

Intro a GLSL

Objetivo

- Familiarizarse con la escritura .
- Entender conceptualmente que un **fragment shader** ejecuta las mismas instrucciones en paralelo para **cada pixel** que tienen una **posicion** y renderiza un **color**.
- Entender por qué los sistemas de referencia son importa

Slides

NOTAS

FRAGMENT SHADER: entra posición sale color.

- Declaración de **variables uniformes:** variables que contienen información "externa" que le programadore le puede pasr. Ejemplo: tiempo.
- void () **main** {...}: cuerpo principal de ejecución.
- **gl_{...}**: son variables provistas para interacturar (leer y escribir información) en distintos momentos de la pipeline gráfica. Ejemplos:
 - gl_FragCoord:** permite leer las coordenadas del píxel.

gl_FragColor: permite escribir el color que se va a renderear.

Introducimos ejes de coordenadas y funciones básicas

Script: Sistemas de referencia

```

float verEjes(vec2 pos){
    float ejes;
    ejes += 1.-step(0.009, distance(pos.x,0.));
    ejes += 1.-step(0.009, distance(pos.y,0.));
    return ejes;
}
void main() {
    vec2 pos; vec3 color;
    pos = uv();
    color.g = verEjes(pos);
    //Visualizamos los ejes
    color.rb = pos; //[( r,b )] == [( x,y )]

    // Mapeamos valores para entender el rango
    //[( r,b )] == [( 1,0 )] full rojo
    //[( r,b )] == [( 0,1 )] full azul
    //[( r,b )] == [( 1.,1. )] full rojo y azul
    //[( r,b )] == [( 0.,0. )] sin rojo, sin azul
    //Identificamos el rango de la variable x y de la variable y

    //////////////////////////////////////
    //Cuento la historia de Descartes y **amamos los sistemas de referencia**

    //Posicionamos el centro de la pantalla en el medio
    //color.rb = uvN(); //[( r,b )] == [( x,y )]

    gl_FragColor = vec4(color,1.);
}

```

Script: tiempo + funciones trigonométricas

```

float verEjes(vec2 pos){
    float ejes;
    ejes += 1.-step(0.009, distance(pos.x,0.));
    ejes += 1.-step(0.009, distance(pos.y,0.));
    return ejes;
}

void main() {
    vec2 pos = uv(); vec3 color;

    //Parte 1. El tiempo al asignar color

    // float freq =10.;
    //color.r = cos(pos.x*freq+time);

    // freq =20.;
    //color.b = cos(pos.x*freq+time);
    //color.g = cos(length(pos)*2.);


    //Parte 2. El tiempo al asignar poscion
    //pos.x +=cos(time);
    //pos.y += sin(time);
    //color += verEjes(pos);
    // color.r += 1.-length(pos);


    gl_FragColor = vec4(color, 1.); // [ (r, g ,b , a) ]

}

```

Script: figuras simples

```

void main() {
    vec2 pos = uv(); vec3 color;
    float scale=1.;
    // Introducimos a las SDF : Funciones de distancias de signo
    //-----
    // Describimos figuras a partir de las posiciones de los pixeles
    float rombo      = smoothstep(0.3,0.31,distance(pos.x,0.)+distance(pos.y*0.5,0.))
    float cuadrado   = smoothstep(0.3,0.31,max(abs(pos.x*.5),abs(pos.y*.5)))-scale*.5
    float circulo    = smoothstep(0.3,0.31,length(pos));
    color.r = rombo;
    color.b = cuadrado;
    color.g = circulo;
    //Introducimos a las SDF : Funciones de distancias de signo
    //-----
    // https://iquilezles.org/www/articles/distfunctions2d/distfunctions2d.htm
    gl_FragColor = vec4(color, 1.); // [ (r, g ,b , a) ]
}

```

NOTAS

- Cada thread "obtiene" información de la posición mediante `gl_FragCoord`
¿Cuál es el valor máximo que puede tomar? ¿Y el mínimo?
- ¿Cómo describimos la posición del píxel? ¿Es única?
 - Sistemas de representación del espacio:

Nombre	Output	Rango	Origen de coordenadas
uvN	(x,y)	[0,1]x[0,1]	esquina inferior izquierda
uv	(x,y)	[-1,1]x[-1,1]	centro de la pantalla

Previous
Presentación

Next
Introducción a mundos 3D

Last updated 1 day ago

Introducción a mundos 3D

Objetivo

- Enteder conceptualmente la técnica de ray marching
- Divertirse dibujando

Slides

Script: raymarching simple



Esfera simple

```

void main () {
    vec3 color;

    vec3 rd = normalize(vec3(uv(),1.5)); // Ray direction
    vec3 ro = vec3(0.,0.,-3.);           // Ray origin, podemos pensarlo donde poner
    float marcha; // Acumulador de las distancias calculadas
    float n;      // Acumulador para crear efecto "neon"

    for (int i=0; i<64; i++)
    {
        vec3 p =ro+marcha*rd;
        float d = length(p)-1.; // SDF para una esfera de radio 1

        if (d < 0.001 || marcha > 50.) {
            break;
        }
        marcha +=d;
        n += 0.1/(.9+abs(d));
    }

    color +=n*0.1;

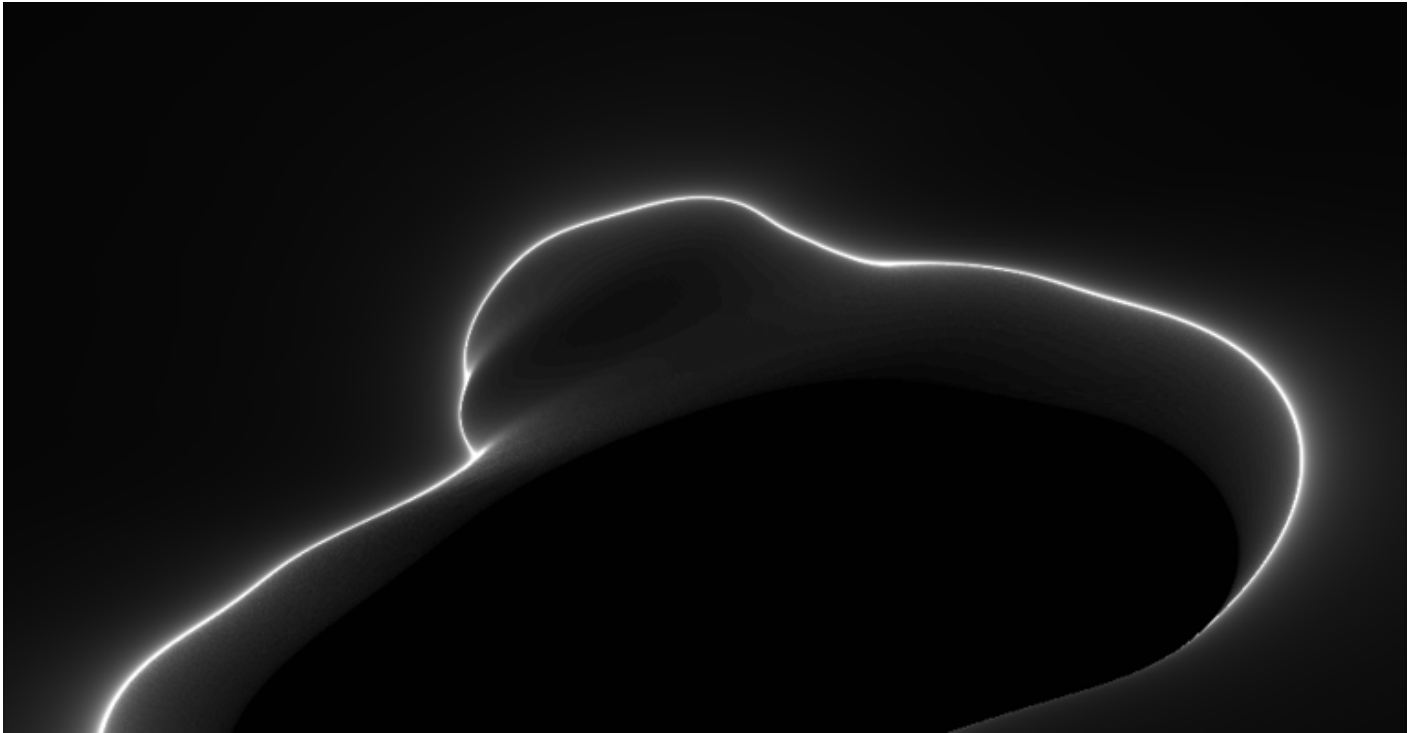
    gl_FragColor = vec4(color, 1.0);
}

```

Notas

- En ray marching, "disparamos" rayos desde la cámara (origen) hacia la escena para determinar lo que la cámara "ve".
- Cada rayo corresponde a un píxel en la pantalla, y su dirección determina hacia dónde apunta en el espacio 3D.
- La dirección del rayo:
 - necesitas proyectar el plano 2D (coordenas uv()) en el espacio 3D para determinar hacia dónde va cada rayo.
 - la **componente z** determina que tan cerca/lejos esta la proyección del espacio 2D.
 - **normalizamos** la dirección del rayo para que las direcciones sean consistentes, es decir, midan lo mismo a la hora de empezar a marchar.

Script: BloBs!



```

float sdEsfera(vec3 p, vec3 offset, float radio){
    return length(p-offset)-radio;
}
float map(vec3 p){
    //pR(p.xz, p.y*1.+time); // Rotacion sobre el eje y
    float d1 = sdEsfera(p, vec3(cos(time)+1., sin(time*2.), 0.),1.);
    float d2 = sdEsfera(p, vec3(cos(time)-1.,sin(time), 0.),1.);
    float d = smin(d1,d2,0.9);
    //d = d + + sin(p.x*10.)*0.03; // Deformaciones sobre el borde
    return d;
}
void main () {
    vec2 pos = uv(); vec3 color;
    vec3 rd = normalize(vec3(pos,1.5));
    vec3 ro = vec3(0.,0.,-10.);
    float marcha;
    float n;

    for (int i=0; i<64; i++)
    {
        vec3 p =ro+marcha*rd;
        float d = map(p);

        if (d < 0.001 || marcha > 50.) {
            break;
        }
        marcha +=d;
        n += 0.1/(.9+abs(d));
    }

    color +=n*0.1;

    gl_FragColor = vec4(color, 1.0);
}

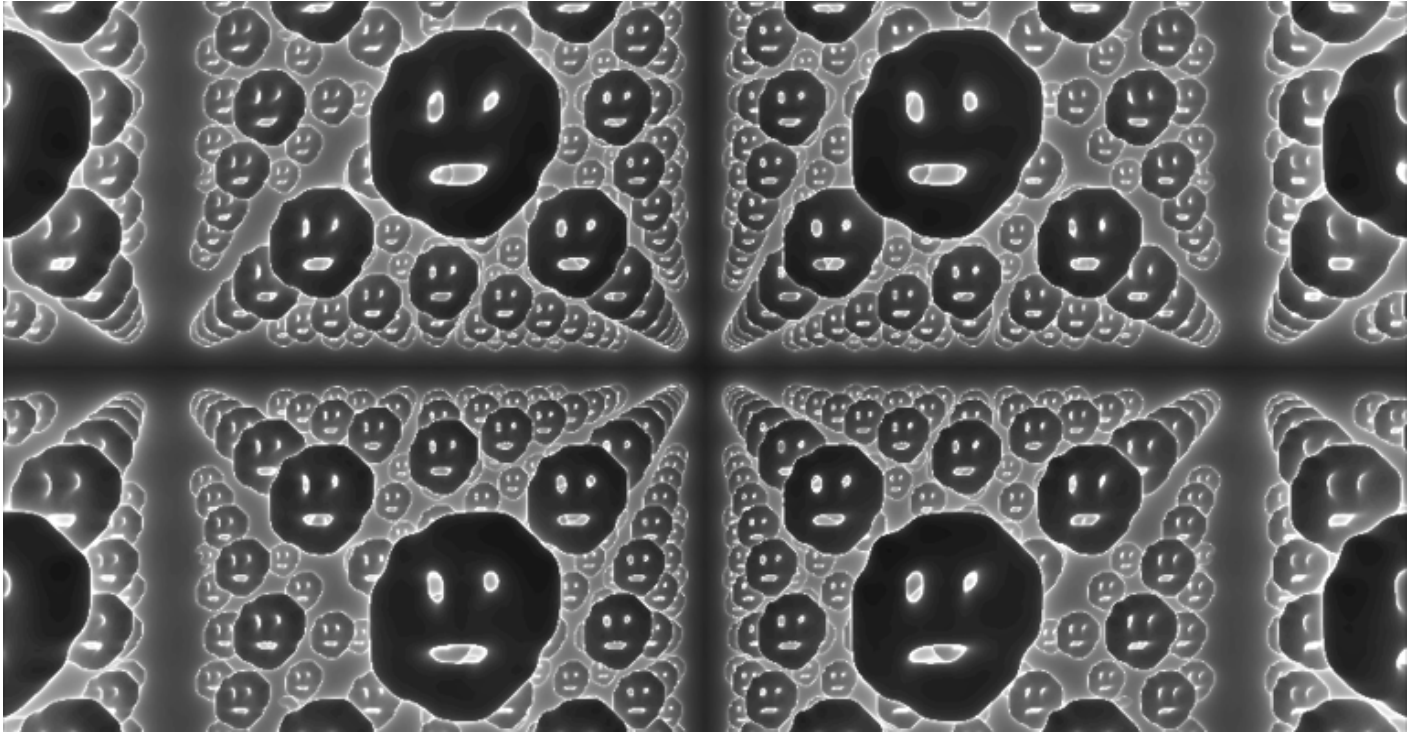
```

Notas

- Vimos como hacer distorciones:
 - rotando al rayo antes de calcular la forma.
 - luego de calcular el valor numérico de la forma, agregar una sinusoidal (**sin()**)
- Para poder describir varios objetos tomamos el mínimo valor entre las distancias de las figuras

- **smin(d1,d2,v)** es una manera suave (smooth) de unir ambas figuras. Qué tan suave sea depende del valor de **v**, a mayor **v** "más unión".

Script: Blobs + Canvas




```

float sdEsfera(vec3 p, vec3 offset, float radio){
    return length(p-offset)-radio;
}

float sdfCanvas(vec3 p){
    float td = length(p-vec3(100.))-0.01;

    for(int i=0; i < 10; ++i){
        for(int j=0; j<10; ++j){
            vec4 s = texture2D(channel0, vec2(float(i)/10.,float(j)/10.));
            float x = mix(-1.,1.,float(i)/10.);
            float y = mix(-1.,1.,float(j)/10.);
            if (s.r>0.){
                float d = sdEsfera(p,vec3(x,y,0.),0.15);
                td = smin(d,td,0.2);
            }

        }
    }
    return td;
}

float map(vec3 p){
    //repeat(p,3.);
    //repeatLimit(p,5.,3.);
    return sdfCanvas(p);
}

void main () {
    vec2 pos = uv(); vec3 color;
    vec3 rd = normalize(vec3(pos,1.5));
    vec3 ro = vec3(0.,0.,-3.);
    float marcha;
    float n;

    for (int i=0; i<64; i++)
    {
        vec3 p =ro+marcha*rd;
        float d = map(p);

        if (d < 0.001 || marcha > 50.) {
            break;
        }
        marcha +=d;
        n += 0.1/(.9+abs(d));
    }

    color +=n*0.2;

    gl_FragColor = vec4(color, 1.0);
}

```

Notas

- En vez de describir la escena a través de definir SDFs matemáticamente, usamos un canvas para dibujar.
 - Leemos de la textura como si fuese una grilla 10×10, cada celda dibuja una esfera en caso de ser pintada de blanco.
 - Usamos las metaballs para dibujar blobs.
- **Bonus:** llamando las funciones **repeat()**, y **repeatLimit()** reproducimos el espacio(sistema de coordenadas) haciendo que nuestra **sdfCanvas** se repita infinitamente, o dentro de límites definidos.

Esto nos permite crear patrones repetitivos o estructuras periódicas en la escena sin necesidad de definir manualmente cada instancia.

Extra

Visualizacion de la funcion que usamos para acumular color

Previous
[Intro a GLSL](#)

Next
[Cámara y Acción](#)

Last updated 15 hours ago